

Promotion Engine System Design

1. Design Summary

ระบบใช้สถาปัตยกรรมแบบ **Modular Monolith** โดยมี:

- Go Fiber Application จำนวน 1 Instance
- MySQL จำนวน 1 Instance
- GORM สำหรับ Database Access
- Promotion Engine แยกออกจาก API และ Database Logic

แนวคิดหลัก:

```
Rule-based Promotion Engine
+ Strategy Pattern
+ Deterministic Calculation Flow
+ Flexible Promotion Database
```

ระบบต้องตอบโจทย์:

1. Promotion ซ้อนกันต้องคำนวณตามลำดับที่ชัดเจน
2. เพิ่ม Promotion ใหม่ได้โดยไม่กระทบ Logic เดิม
3. รองรับ Promotion รูปแบบใหม่โดยไม่แก้ Database Schema บ่อย
4. คำนวณราคาอย่างถูกต้องและตรวจสอบย้อนหลังได้

2. Overall System Architecture

flowchart LR

Client["Client / Tester"]

subgraph Application["Go Fiber Application - Single Insta"]

```

nce"]
    Handler["HTTP Handler<br/>Validate Request / Return R
esponse"]
    Service["Pricing Service<br/>ควบคุม Calculation Use Ca
se"]
    Engine["Promotion Engine<br/>Core Calculation Logic"]
    Repository["GORM Repository<br/>Database Access"]
end

MySQL[("MySQL Database")]

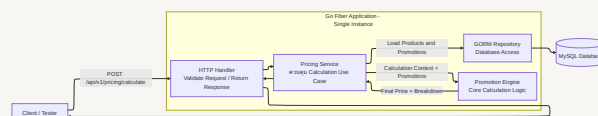
Client -->|"POST /api/v1/pricing/calculate"| Handler
Handler --> Service

Service -->|"Load Products and Promotions"| Repository
Repository --> MySQL

Service -->|"Calculation Context + Promotions"| Engine
Engine -->|"Final Price + Breakdown"| Service

Service --> Handler
Handler --> Client

```



Component Responsibilities

Component	Responsibility
HTTP Handler	รับ Request, Validate เบื้องต้น และ Return Response
Pricing Service	โหลดข้อมูลและควบคุม Use Case
Promotion Engine	ตรวจ Rule, เรียงลำดับ และ Apply Promotion
GORM Repository	โหลด Product และ Promotion จาก MySQL

Component	Responsibility
MySQL	เก็บ Product, Promotion Config และ Calculation Log

3. Promotion Engine Design Pattern

```

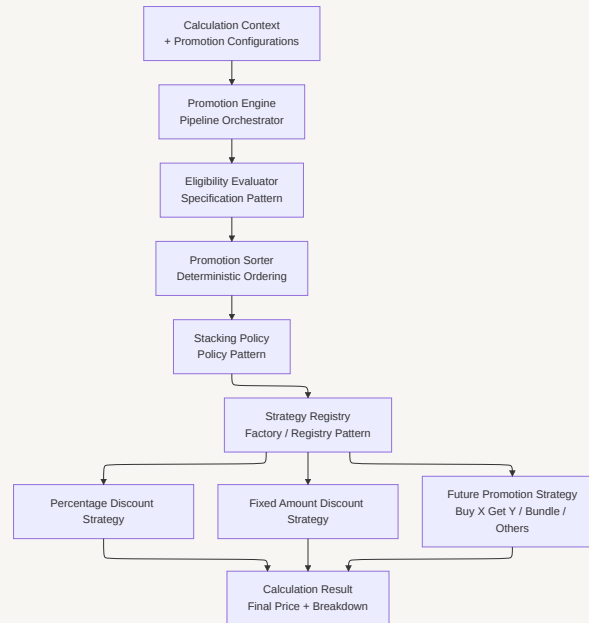
flowchart TD
    Input["Calculation Context<br/>+ Promotion Configuration  
s"]
    Engine["Promotion Engine<br/>Pipeline Orchestrator"]
    Eligibility["Eligibility Evaluator<br/>Specification Pattern"]
    Sorter["Promotion Sorter<br/>Deterministic Ordering"]
    Policy["Stacking Policy<br/>Policy Pattern"]
    Registry["Strategy Registry<br/>Factory / Registry Pattern"]
    Percentage["Percentage Discount Strategy"]
    Fixed["Fixed Amount Discount Strategy"]
    Future["Future Promotion Strategy<br/>Buy X Get Y / Bundle / Others"]
    Result["Calculation Result<br/>Final Price + Breakdown"]

    Input --> Engine
    Engine --> Eligibility
    Eligibility --> Sorter
    Sorter --> Policy
    Policy --> Registry
    Registry --> Percentage
    Registry --> Fixed
    Registry --> Future
    Percentage --> Result
    Fixed --> Result
    Future --> Result
  
```

Percentage --> Result

Fixed --> Result

Future --> Result



Design Patterns ที่ใช้

Pattern	หน้าที่	เหตุผล
Strategy Pattern	แยกสูตรคำนวณ Promotion แต่ละประเภท	เพิ่ม Promotion ใหม่โดยไม่แก้ Logic เดิม
Factory/Registry Pattern	เลือก Strategy จาก <code>action_type</code>	ลด If-Else ใน Promotion Engine
Specification Pattern	ตรวจสอบ Condition และ Target	แยก Eligibility ออกจากสูตรคำนวณ
Pipeline Pattern	Apply Promotion ตามลำดับ	รองรับ Promotion ซ้อนกัน
Policy Pattern	ควบคุม Stacking และ Conflict	แยก Business Policy ออกจาก Strategy
Repository Pattern	แยก Database Access	Engine ไม่ผูกกับ GORM/MySQL
Context Object	เก็บ State ระหว่างคำนวณ	สร้าง Breakdown และ Debug ได้

4. Promotion Calculation Flow

flowchart TD

```
Start([รับ Order Items]) --> Validate[Validate Request]
Validate --> LoadProducts[โหลดราคาสินค้าจาก Server]
LoadProducts --> BuildContext[สร้าง Calculation Context]
BuildContext --> LoadPromotions[โหลด Active Promotions]
LoadPromotions --> Evaluate[ตรวจ Condition และ Target]
Evaluate --> Sort[เรียง Promotion ตาม Scope และ Priority]
```

```
Sort --> Next[เลือก Promotion ตัวถัดไป]
```

```
Next --> Stopped{Target ถูกหยุดแล้วหรือไม่}
```

```
Stopped -- Yes --> SkipStopped[Skip: scope_stopped]
```

```
Stopped -- No --> Stack{Stacking Policy ผ่านหรือไม่}
```

```
Stack -- No --> SkipConflict[Skip: stacking_conflict]
```

```
Stack -- Yes --> Strategy{พบ Strategy หรือไม่}
```

```
Strategy -- No --> SkipStrategy[Skip: strategy_not_found]
```

```
Strategy -- Yes --> Calculate[คำนวณ Discount]
```

```
Calculate --> Cap[Cap Discount ไม่ให้เกินยอดคงเหลือ]
```

```
Cap --> Apply[Apply Discount]
```

```
Apply --> Breakdown[บันทึก Before / Discount / After]
```

```
Breakdown --> Stop{Exclusive หรือ Stop Processing?}
```

```
Stop -- Yes --> MarkStopped[หยุด Promotion ที่ขัดแย้ง]
```

```
Stop -- No --> More{มี Promotion เหลือหรือไม่}
```

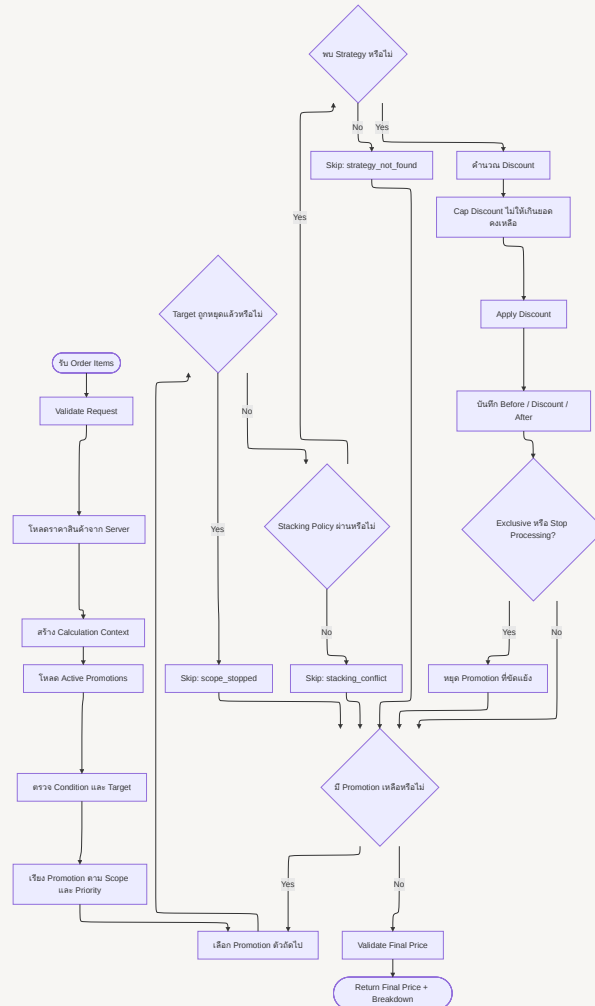
```
MarkStopped --> More
```

```
SkipStopped --> More
```

```
SkipConflict --> More
```

```
SkipStrategy --> More
```

```
Final --> Result([Return Final Price + Breakdown])
```



ITEM → CART → COUPON → SHIPPING → ROUNDING

```
priority ASC
→ created_at ASC
```

→ promotion_id ASC

`priority` คำน้อยกว่าค่านวนก่อน

5. Promotion Stacking Rules

Configuration	Behavior
<code>priority</code>	กำหนด Promotion ที่ค่านวนก่อน
<code>stackable=true</code>	ใช้ร่วมกับ Promotion อื่นได้
<code>stackable=false</code>	Skip เมื่อมี Promotion ที่ขัดแย้งถูกใช้แล้ว
<code>exclusive=true</code>	Apply แล้วหยุด Promotion อื่นใน Conflict Group เดียวกัน
<code>stop_processing=true</code>	Apply แล้วหยุด Rule ถัดไปบน Target เดียวกัน

ตัวอย่าง Promotion ซ้อนกัน

Product 1 ราคา 1,000 บาท

Promotion A:

ลด 10%

`priority = 1`

`stackable = true`

Promotion B:

ลด 100 บาท

`priority = 2`

`stackable = true`

คำนวณแบบ Sequential จากยอดคงเหลือ:

เริ่มต้น	= 1,000 บาท
Promotion A ลด 10%	= -100 บาท
ยอดคงเหลือ	= 900 บาท
Promotion B ลด 100 บาท	= -100 บาท
ราคาสุทธิ	= 800 บาท

ถ้าสลับ Priority:

```
1,000 - 100 = 900
900 - 10% = 810
```

ดังนั้น Priority และ Tie-breaker ต้องชัดเจนเพื่อให้ระบบเป็น **Deterministic**

6. Flexible Database Design

erDiagram

```
PRODUCTS ||--o{ ORDER_ITEMS : referenced_by
ORDERS ||--|{ ORDER_ITEMS : contains
```

```
PROMOTIONS ||--o{ PROMOTION_TARGETS : targets
PROMOTIONS ||--o{ PROMOTION_CONDITIONS : has
PROMOTIONS ||--|{ PROMOTION_ACTIONS : performs
PROMOTIONS ||--o{ PROMOTION_USAGES : consumed_by
```

```
ORDERS ||--o{ PROMOTION_USAGES : records
ORDERS ||--o{ CALCULATION_LOGS : audited_by
```

```
PRODUCTS {
    bigint id PK
    varchar sku UK
    varchar name
    bigint price_amount
    varchar currency
    varchar status
}
```

```
ORDERS {
    bigint id PK
    varchar order_no UK
    bigint original_total
    bigint discount_total
    bigint final_total
```



```

        varchar currency
        varchar status
    }

ORDER_ITEMS {
    bigint id PK
    bigint order_id FK
    bigint product_id FK
    int quantity
    bigint unit_price
    bigint original_amount
    bigint discount_amount
    bigint final_amount
}

PROMOTIONS {
    bigint id PK
    varchar code UK
    varchar name
    varchar scope
    int priority
    boolean stackable
    boolean exclusive
    boolean stop_processing
    varchar conflict_group
    varchar status
    datetime starts_at
    datetime ends_at
}

PROMOTION_TARGETS {
    bigint id PK
    bigint promotion_id FK
    varchar target_type
    bigint target_id
}

```

```

PROMOTION_CONDITIONS {
    bigint id PK
    bigint promotion_id FK
    varchar condition_type
    varchar operator
    json value_json
}

PROMOTION_ACTIONS {
    bigint id PK
    bigint promotion_id FK
    varchar action_type
    bigint value_amount
    int value_basis_points
    bigint max_discount_amount
    json value_json
}

PROMOTION_USAGES {
    bigint id PK
    bigint promotion_id FK
    bigint user_id
    bigint order_id FK
    int usage_count
}

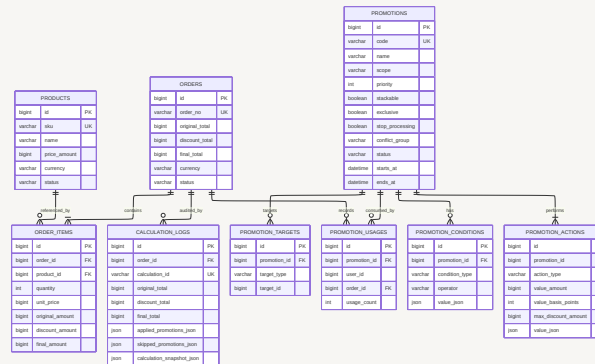
CALCULATION_LOGS {
    bigint id PK
    bigint order_id FK
    varchar calculation_id UK
    bigint original_total
    bigint discount_total
    bigint final_total
    json applied_promotions_json
    json skipped_promotions_json
}

```

```

}
    json calculation_snapshot_json
}

```



Flexible Schema Concept

Promotion

- └ Metadata: Promotion คืออะไรและคำนวณลำดับใด
- └ Target: ใช้กับสินค้าใด
- └ Condition: ใช้ได้เมื่อใด
- └ Action: ลดอย่างไร

Table	หน้าที่
promotions	เก็บข้อมูลหลัก Priority และ Stacking Policy
promotion_targets	ระบุ Product หรือ Category ที่ได้รับส่วนลด
promotion_conditions	ระบุเงื่อนไขการใช้งาน
promotion_actions	ระบุสูตรคำนวณส่วนลด
promotion_usages	เก็บการใช้ Promotion จริง
calculation_logs	เก็บ Breakdown และ Audit Snapshot

7. ตัวอย่างข้อมูลตามโจทย์

Product 1 ลด 10%

```
promotions:
  id = 1
  code = ITEM1_10_PERCENT
  scope = ITEM
  priority = 1
  stackable = true
  status = ACTIVE

promotion_targets:
  promotion_id = 1
  target_type = PRODUCT
  target_id = 1

promotion_actions:
  promotion_id = 1
  action_type = PERCENTAGE_DISCOUNT
  value_basis_points = 1000
```

Product 2 ลด 100 บาท

```
promotions:
  id = 2
  code = ITEM2_MINUS_100
  scope = ITEM
  priority = 1
  stackable = true
  status = ACTIVE

promotion_targets:
  promotion_id = 2
  target_type = PRODUCT
  target_id = 2

promotion_actions:
  promotion_id = 2
  action_type = FIXED_AMOUNT_DISCOUNT
  value_amount = 10000
```

10000 หมายถึง 100 บาท เนื่องจากเก็บเงินเป็นหน่วย satang

8. Correctness Rules

Money Design

```
ใช้ BIGINT หน่วย satang  
100 บาท = 10,000  
1,000 บาท = 100,000
```

Percentage ใช้ Basis Points:

```
10% = 1,000 basis points  
100% = 10,000 basis points
```

ห้ามใช้ Floating Point กับเงิน

Calculation Guard

```
actual_discount = min(calculated_discount, remaining_amount)  
final_amount = remaining_amount - actual_discount  
final_amount >= 0
```

ตัวอย่าง Discount เกินราคา

```
ราคาสินค้า = 80 บาท  
ส่วนลด = 100 บาท  
  
Actual Discount = 80 บาท  
Final Price = 0 บาท
```

Calculation Breakdown

ทุก Promotion ต้องบันทึก:

```
promotion_id  
promotion_code  
before_amount  
discount_amount  
after_amount  
status = APPLIED / SKIPPED
```

```
skipped_reason  
execution_order
```

9. End-to-End Sequence Diagram

```
sequenceDiagram
    actor Client
    participant API as Go Fiber Handler
    participant Service as Pricing Service
    participant ProductRepo as Product Repository
    participant PromoRepo as Promotion Repository
    participant Engine as Promotion Engine
    participant LogRepo as Calculation Log Repository
    participant DB as MySQL

    Client->>API: POST /api/v1/pricing/calculate
    API->>API: Validate request
    API->>Service: Calculate request

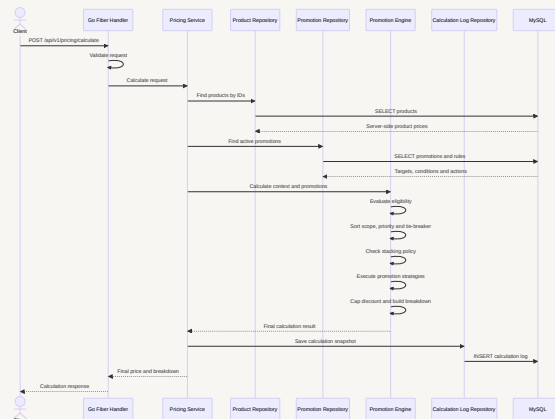
    Service->>ProductRepo: Find products by IDs
    ProductRepo->>DB: SELECT products
    DB-->>ProductRepo: Server-side product prices

    Service->>PromoRepo: Find active promotions
    PromoRepo->>DB: SELECT promotions and rules
    DB-->>PromoRepo: Targets, conditions and actions

    Service->>Engine: Calculate context and promotions
    Engine->>Engine: Evaluate eligibility
    Engine->>Engine: Sort scope, priority and tie-breaker
    Engine->>Engine: Check stacking policy
    Engine->>Engine: Execute promotion strategies
    Engine->>Engine: Cap discount and build breakdown
    Engine-->>Service: Final calculation result
```

Service->>LogRepo: Save calculation snapshot
LogRepo->>DB: INSERT calculation log

Service-->>API: Final price and breakdown
API-->>Client: Calculation response



10. การเพิ่ม Promotion ใหม่

ตัวอย่างเพิ่ม `BUY_X_GET_Y`

Database

```
{
  "action_type": "BUY_X_GET_Y",
  "value_json": {
    "buy_quantity": 2,
    "free_quantity": 1
  }
}
```

Application

1. เพิ่ม BuyXGetYStrategy
2. Register เข้า Strategy Registry
3. เพิ่ม Validation สำหรับ value_json
4. เพิ่ม Unit Test และ Regression Test

ไม่ต้องแก้:

- Promotion Engine Pipeline
- Pricing Service
- Percentage Strategy
- Fixed Amount Strategy
- Database Table หลัก

11. คำตอบตรงคำถาม

Promotion ซ่อนกันจัดลำดับอย่างไร

เรียงตาม:
scope_order ASC
→ priority ASC
→ created_at ASC
→ promotion_id ASC

จากนั้นคำนวณ Sequential จากยอดคงเหลือ
และควบคุมด้วย stackable, exclusive, stop_processing

เพิ่ม Promotion โดยไม่กระทบ Logic เดิมอย่างไร

ใช้ Strategy Pattern แยกสูตรคำนวณ
ใช้ Strategy Registry เลือก Strategy จาก action_type
เพิ่ม Strategy ใหม่โดยไม่แก้ Promotion Engine Flow

รองรับ Promotion ใหม่ที่ไม่เคยมีมาก่อนอย่างไร

Database รองรับด้วย action_type และ value_json
Condition ใหม่เพิ่มผ่าน condition_type และ Specification ใหม่
Application เพิ่ม Strategy และ Validation ใหม่

12. Requirement Coverage

Requirement	Design Solution	Status
Product 1 ลด 10%	Percentage Discount Strategy	ครบ
Product 2 ลด 100 บาท	Fixed Amount Strategy	ครบ
Promotion ซ้อนกัน	Priority + Stacking Policy + Sequential Calculation	ครบ
เพิ่ม Promotion ไม่กระทบ Logic เดิม	Strategy + Registry	ครบ
Promotion ใหม่ที่ไม่เคยมี	Config-driven Action + New Strategy	ครบ
Design Pattern	Strategy, Factory, Specification, Pipeline, Policy, Repository	ครบ
Flexible Table	Promotion + Target + Condition + Action	ครบ
คำนวณถูกต้อง	Deterministic Sort + Discount Cap + Breakdown	ครบ

13. Business Rules ที่ต้องตกลงก่อนพัฒนา

เพื่อไม่ให้เกิดความคลุมเครือ ต้องกำหนดเพิ่ม:

Rule	ข้อเสนอสำหรับ MVP
Non-stackable Conflict	Priority First
Percentage Base	คำนวณจาก Remaining Amount
Priority เท่ากัน	created_at ASC → promotion_id ASC
Exclusive Scope	หยุด Conflict Group บน Target เดียวกัน
Stop Processing	หยุด Rule ถัดไปบน Target เดียวกัน
Rounding	ใช้ Integer Division แบบ Floor

Rule	ข้อเสนอสำหรับ MVP
Client Price	ไม่รับเป็น Source of Truth
Final Price	ห้ามต่ำกว่า 0

Final Summary

Design นี้ตอบโจทย์ครบโดยใช้:

```
Single Go Fiber Modular Monolith
+ Promotion Engine
+ Strategy Pattern
+ Deterministic Calculation Pipeline
+ Stacking Policy
+ Rule-based Flexible Database
+ Calculation Breakdown
```

หัวใจสำคัญคือ Promotion Engine ควบคุมเฉพาะ Flow ส่วนสูตรคำนวณแต่ละ Promotion แยกเป็น Strategy และ Promotion Configuration แยกเป็น Metadata, Target, Condition และ Action ทำให้รองรับ Promotion ซ้อนกัน เพิ่ม Promotion ใหม่ได้ และคำนวณราคาได้อย่างถูกต้องครับ